

Sprawozdanie: Przybliżanie języka naturalnego - kontynuacja

Jakub Bilski 155865
Teoria Informacji i Metody Kompresji Danych - lab2

17 maja 2025

1 Wprowadzenie

Celem sprawozdania jest:

- analiza statystyk słów w korpusie tekstu angielskiego,
- obliczenie pokrycia korpusu przez najczęstsze słowa,
- implementacja i porównanie:
 - modelu zerowego (unigram),
 - bigramu według metody Shannon'a,
 - modeli Markowa I i II rzędu.

2 Przygotowanie danych

1. Wczytanie tekstu z pliku `norm_wiki_sample.txt` i konwersja na małe litery.
2. Tokenizacja wzorcem `[a-z']+` – wyodrębnienie słów z apostrofami.
3. Zapis wyników:
 - liczba tokenów: `total_tokens`,
 - liczba unikalnych typów: `unique_types`.

3 Kod i wyniki

3.1 Obliczanie pokrycia i najczęstszych słów

```
import re
from collections import Counter

# Przygotowanie korpusu
with open('norm_wiki_sample.txt', 'r', encoding='utf-8') as f:
    text = f.read().lower()
words = re.findall(r"[a-z']+", text)
total_tokens = len(words)

# Statystyki
freq = Counter(words)

def coverage(n):
    topn = sum(c for _, c in freq.most_common(n))
```

```

        return topn / total_tokens * 100

cov_6000 = coverage(6000)
cov_30000 = coverage(30000)
print(f"Coverage top 6000 words: {cov_6000:.2f}%")
print(f"Coverage top 30000 words: {cov_30000:.2f}%\n")

# Top 10 s    w
top10 = freq.most_common(10)
print("Top 10 s    w (word, count, %):")
for w,c in top10:
    print(f"{w:<12}{c:>8}{c/total_tokens*100:>8.2f}%")

```

Listing 1: Statystyki częstości i pokrycie top- N .

Otrzymane wyniki:

Coverage top 6000 words: 83.14%
 Coverage top 30000 words: 95.29%

Top 10 słów (word, count, %):

the	118993	6.81%
of	59074	3.38%
and	48809	2.79%
in	47677	2.73%
a	37096	2.12%
to	34015	1.95%
was	19579	1.12%
is	16651	0.95%
for	14178	0.81%
on	13896	0.80%

Interpretacja:

- **Pokrycie 6 000:** 83 % oznacza, że niewielki zbiór słów wystarcza do objęcia większości tekstu.
- **Pokrycie 30 000:** 95 % – dalszy wzrost, lecz malejące przyrosty wyniku.
- **Top 10:** dominują często funkcje gramatyczne (rodzajniki, przyimki), co jest zgodne z prawem Zipfa.

3.2 Model zerowy (unigram)

```

import random

vocab, counts = zip(*freq.items())
probs = [c/total_tokens for c in counts]

def generate_unigram(n):
    return random.choices(vocab, weights=probs, k=n)

print(generate_unigram(20))

```

Listing 2: Generacja unigramowa (model zerowy).

Przykładowy ciąg:

company run he in extreme of sheep to non days the stops them progressive and a entering fe

Komentarz: brak spójności semantycznej ani składniowej – model ignoruje kontekst.

3.3 Metoda Shannon’a (bigram)

```
def generate_shannon_bigram(start, length):
    seq = [start]
    for _ in range(length-1):
        pos = [i for i,w in enumerate(words[:-1]) if w==seq[-1]]
        if not pos:
            seq.append(random.choice(vocab))
        else:
            idx = random.choice(pos)
            seq.append(words[idx+1])
    return seq

print(generate_shannon_bigram('probability',20))
```

Listing 3: Sampling bezpośrednich następców (Shannon).

Przykład (start "probability"):

probability that date of local life technological positions over the guelph victory while re

Komentarz:

- Generuje realne kolokacje z korpusu.
- Ograniczenie: nie tworzy nowych kombinacji, jedynie odtwarza istniejące sekwencje.

3.4 Model Markowa I rzędu (bigram probabilistyczny)

```
from collections import defaultdict, Counter

# Zliczanie par
transition1 = defaultdict(Counter)
for prev,curr in zip(words, words[1:]):
    transition1[prev][curr] += 1

# Normalizacja
probs1 = {
    prev: {w:cnt/sum(c.values()) for w,cnt in counter.items()}
    for prev,counter in transition1.items()
}

def generate_markov_bigram(start, length):
    seq = [start]
    for _ in range(length-1):
        prev = seq[-1]
        if prev not in probs1:
            seq.append(random.choices(vocab,weights=probs)[0])
        else:
            choices,weights = zip(*probs1[prev].items())
```

```

        seq.append(random.choices(choices, weights=weights)[0])
    return seq

print(generate_markov_bigram('probability', 20))

```

Listing 4: Budowa tablicy i generacja bigramowa.

Przykład:

probability of the same reputation as to the pacific karen karen leaves him has proposed in

Komentarz:

- Zachowuje statystyczne rozkłady warunkowe.
- Umożliwia tworzenie nowych sekwencji, a nie tylko odtwarzanie.
- Fallback do unigramu zabezpiecza przed nieznanymi słowami.

3.5 Model Markowa II rzędu (trigram probabilistyczny)

```

# Zliczenie trigram w
transition2 = defaultdict(Counter)
for w1,w2,w3 in zip(words, words[1:], words[2:]):
    transition2[(w1,w2)][w3] += 1

# Normalizacja
probs2 = {
    pair: {w: cnt/sum(c.values()) for w, cnt in counter.items()}
    for pair, counter in transition2.items()
}

def generate_markov_trigram(start_pair, length):
    seq = list(start_pair)
    for _ in range(length-2):
        pair = (seq[-2], seq[-1])
        if pair not in probs2:
            seq.append(random.choices(vocab, weights=probs)[0])
        else:
            choices, weights = zip(*probs2[pair].items())
            seq.append(random.choices(choices, weights=weights)[0])
    return seq

print(generate_markov_trigram(('probability', 'of'), 20))

```

Listing 5: Budowa tablicy i generacja trigramowa.

Przykład:

probability of in addition to molecular stability conformational analysis one of whistlers b

Komentarz:

- Lepiej oddaje krótkie zależności składniowe.
- Rośnie liczba parametrów — wymaga więcej danych.

4 Podsumowanie wniosków

- **Prawo Zipfa:** kilka słów odpowiada za znaczną część korpusu.
- **Unigram:** prosty, ale bez kontekstu – niska koherencja.
- **Shannon:** wierne odtwarzanie kolokacji, brak generalizacji.
- **Markow:** probabilistyczne uogólnienie – równowaga między spójnością a kreatywnością.
- **Trigram:** lepsza koherencja składniowa, kosztem większej złożoności modelu.